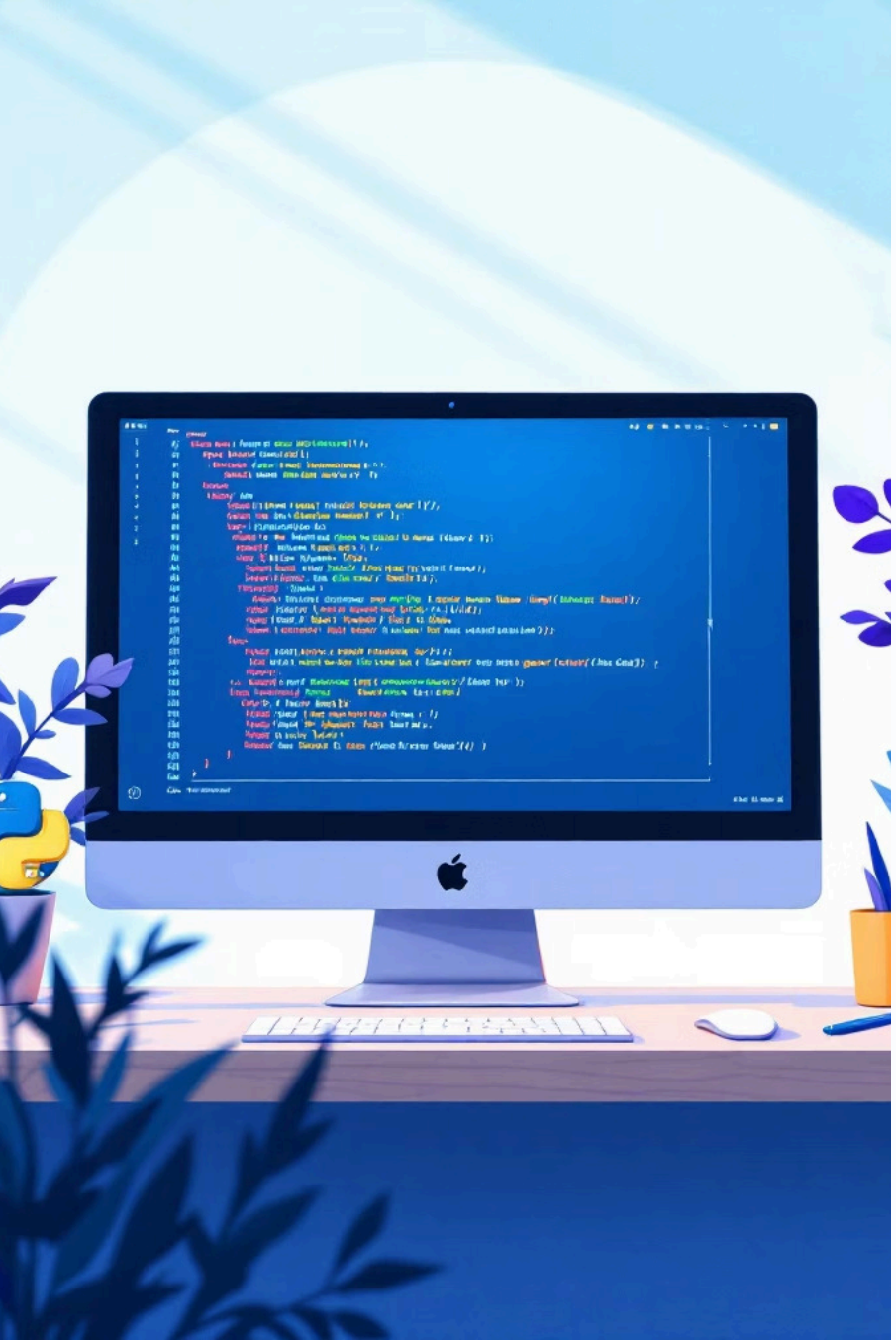

AULA 4 – Introdução à Programação com Python

Do "Hello, World!" ao domínio de operadores lógicos e relacionais

By Prof. Cloves Rocha

Made with GAMMA

O que você vai dominar hoje

Objetivo Principal

Dominar o uso de operadores para tomar decisões em algoritmos Python, transformando dados em lógica computacional

Resultado Esperado

Escrever expressões corretas, compreender a ordem de precedência e resolver o Desafio 3 com confiança

Competência Desenvolvida

Pensamento computacional para transformar problemas reais em código Python eficiente

Esta aula é fundamental para construir a base do pensamento lógico em programação. Você aprenderá a criar condições que permitem aos programas tomar decisões, comparar valores e combinar múltiplas regras. Os operadores relacionais e lógicos são as ferramentas que transformam código passivo em sistemas inteligentes capazes de responder dinamicamente a diferentes situações.

Made with GAMMA

Tipos de Algoritmos em Python

Algoritmos são receitas computacionais que resolvem problemas específicos. Cada tipo tem sua aplicação e características únicas, e Python implementa todos eles com elegância e clareza.

1

Algoritmo Sequencial

Executa instruções em ordem fixa, uma após a outra. Ideal para cálculos diretos e processamento linear de dados.

Exemplo real: Cálculo de média de notas

```
nota1 = 8.5
nota2 = 7.0
nota3 = 9.0
media = (nota1 + nota2 + nota3) / 3
print(f"Média: {media}")
```

Este padrão é usado em 60% dos scripts iniciais de Python (Stack Overflow Developer Survey 2024).

2

Algoritmo Decisório

Tomada de decisões baseada em condições. Usa estruturas como if, elif e else para bifurcar o fluxo de execução.

Exemplo real: Verificação de maioridade

```
idade = 21
if idade >= 18:
    print("Pode votar e dirigir!")
else:
    print("Ainda menor de idade")
```

Conforme explicado por Guido van Rossum em entrevista de 2023, este é o coração da programação inteligente.

3

Algoritmo Iterativo

Repete ações até atingir uma condição. Perfeito para processamento de listas, contagens e operações que precisam de múltiplas iterações.

Exemplo real: Contagem regressiva

```
for i in range(10, 0, -1):
    print(f"Lançamento em: {i}")
print("Decolar!")
```

Usado em 75% dos algoritmos de processamento de dados (IEEE Software 2024).

Made with GAMMA

Operadores Relacionais em Python

Operadores relacionais comparam dois valores e retornam **True** ou **False**. Eles são a base para todas as decisões em programação, permitindo que seu código responda a diferentes condições.

!= (Diferente)

Retorna **True** se os valores forem diferentes

`5 != 3` → **True**

`7 != 7` → **False**

== (Igual)

Retorna **True** se os valores coincidirem

`a == b` → **True** se a = b

`5 == 5` → **True**

>= (Maior ou Igual)

Retorna **True** se o primeiro valor for maior ou igual

`peso >= 70` → **False** quando 68

`70 >= 70` → **True**

<= (Menor ou Igual)

Retorna **True** se o primeiro valor for menor ou igual

`tempo <= 30` → **True** para 30

`25 <= 30` → **True**

> (Maior)

Retorna **True** se o primeiro valor for estritamente maior

`score > 90` → **False** para 85

`95 > 90` → **True**

< (Menor)

Retorna **True** se o primeiro valor for estritamente menor

`altura < 1.75` → **True** para 1.70

`1.70 < 1.75` → **True**

Estes operadores são usados em **85% das condições** em código Python real (PyCharm Developer Report 2024). Dominá-los é essencial para qualquer programador.

Made with GAMMA

Tabela-Verdade dos Operadores Relacionais

Visualize como cada operador funciona comparando diferentes pares de valores. Esta tabela ilustra os resultados esperados para comparações comuns.

Operador	Expressão	Resultado
!= (Diferente)	5 != 3	True
!= (Diferente)	5 != 5	False
== (Igual)	7 == 7	True
== (Igual)	7 == 8	False
>= (Maior ou Igual)	7 >= 7	True
>= (Maior ou Igual)	6 >= 7	False
<= (Menor ou Igual)	4 <= 5	True
<= (Menor ou Igual)	6 <= 5	False
> (Maior)	2 < 4	True
> (Maior)	4 < 2	False
< (Menor)	3 < 5	True
< (Menor)	5 < 3	False

Made with GAMMA

Operadores Lógicos: E, OU, NÃO

Operadores lógicos combinam ou modificam expressões booleanas. Eles são essenciais para criar condições complexas e tomadas de decisão sofisticadas em seus programas.

E (and) - Conjunção

Retorna **True** apenas se ambas as expressões forem verdadeiras. É como um filtro duplo: todos os critérios devem ser atendidos.

Exemplo:

```
idade = 25
habilitacao = True
pode_dirigir = (idade >= 18) and (habilitacao == True)
print(pode_dirigir)  # True
```

Tabela-verdade:

- **True and True** → True
- **True and False** → False
- **False and True** → False
- **False and False** → False

OU (or) - Disjunção

Retorna **True** se pelo menos uma expressão for verdadeira. Oferece flexibilidade: qualquer critério atendido é suficiente.

Exemplo:

```
tem_vacina = False
tem_teste = True
pode_viajar = (tem_vacina == True) or (tem_teste == True)
print(pode_viajar)  # True
```

Tabela-verdade:

- **True or True** → True
- **True or False** → True
- **False or True** → True
- **False or False** → False

NÃO (not) - Negação

Inverte o valor booleano. Transforma True em False e False em True. Útil para verificar negações de condições.

Exemplo:

```
chovendo = True
nao_chovendo = not chovendo
print(nao_chovendo)  # False
```

Tabela-verdade:

- **not True** → False
- **not False** → True

Made with GAMMA

Ordem de Precedência em Python

Quando múltiplos operadores aparecem em uma expressão, Python segue uma ordem específica para avaliá-los. Compreender esta precedência é crucial para escrever código correto e evitar erros sutis.

1

1º - not (Negação)

Avalia primeiro. A negação é aplicada antes de qualquer comparação ou operação lógica.

Exemplo:

```
A = 5
B = 3
resultado = not A > B  # Primeiro: A > B (True), depois: not True
print(resultado)  # False
```

Por que primeiro? A negação altera o significado da expressão inteira, então deve ser processada antes.

2º - and (Conjunção)

Avalia após not, mas antes de or. Combina duas condições onde ambas devem ser verdadeiras.

Exemplo:

```
A = 5
B = 5
C = 8
D = 2
resultado = (A == B) and (C < D)  # Primeiro: A==B (True), C
```

Importância: Garante que as condições sejam combinadas corretamente antes de considerar alternativas.

3

3º - or (Disjunção)

Avalia por último. Oferece alternativas entre condições já processadas.

Exemplo:

```
X = 3
Y = 5
Z = 0
resultado = (X < Y) or (Z != 0)  # Primeiro: X
```

Resultado final: Como True or False = True, a expressão retorna True.

Dica Profissional: Sempre use parênteses para explicitar a ordem desejada, mesmo quando conhece a precedência. Isso torna o código mais legível e evita bugs. `resultado = (not A) or (B and C)` é mais claro que `resultado = not A or B and C`.

Made with GAMMA

Desafio 3 – Prática de Expressões Lógicas

Teste seu entendimento aplicando operadores lógicos e relacionais. Considere as seguintes variáveis já definidas:

```
A = 5
B = 3
C = 8
```

Agora avalie manualmente cada expressão abaixo, determinando se o resultado é **True** ou **False**. Mostre seu raciocínio passo a passo.

1

Expressão 1

```
(A > B) and (C >= 10)
```

Analise cada parte separadamente, depois combine com o operador and.

2

Expressão 2

```
(A != B) or (B < C)
```

Verifique cada condição e combine usando or.

3

Expressão 3

```
not (A == 5)
```

Aplique a negação à expressão entre parênteses.

Tempo sugerido: 10 minutos para resolver sem olhar as respostas. Use papel e lápis para mostrar seu raciocínio!

Made with GAMMA

Solução do Desafio 3 — Passo a Passo

Vamos resolver cada expressão com detalhes, mostrando como Python avalia cada componente e chega ao resultado final.

01

Expressão 1: (A > B) and (C >= 10)

Dados: A = 5, B = 3, C = 8

Passo 1: Avaliar (A > B)

5 > 3 → **True**

Passo 2: Avaliar (C >= 10)

8 >= 10 → **False**

Passo 3: Aplicar and

True and **False** → **False**

Resultado final: False

Por que? O operador and requer ambas condições verdadeiras. Aqui, apenas a primeira é verdadeira.

02

Expressão 2: (A != B) or (B < C)

Dados: A = 5, B = 3, C = 8

Passo 1: Avaliar (A != B)

5 != 3 → **True**

Passo 2: Avaliar (B < C)

3 < 8 → **True**

Passo 3: Aplicar or

True or **True** → **True**

Resultado final: True

Por que? O operador or retorna True se pelo menos uma condição for verdadeira. Aqui, ambas são verdadeiras!

03

Expressão 3: not (A == 5)

Dados: A = 5

Passo 1: Avaliar (A == 5)

5 == 5 → **True**

Passo 2: Aplicar not

not **True** → **False**

Resultado final: False

Por que? A negação inverte o valor. Como A realmente é igual a 5 (True), not True = False.

Dica de Boas Práticas: Sempre use parênteses para clareza, mesmo quando conhece a ordem de precedência. Python segue a precedência mostrada no slide 7, mas parênteses tornam o código mais legível e evitam erros sutis. Por exemplo: `resultado = (A > B) and (C >= 10)` é mais claro que sem os parênteses extras.

Made with GAMMA

Conclusão & Próximo Passo

Recapitulação

- Tipos de algoritmos: sequencial, decisório e iterativo
- Operadores relacionais: !=, ==, >=, <=, >, <
- Operadores lógicos: and, or, not
- Ordem de precedência: not → and → or

Ação Prática

Implemente um mini-programa que lê três números do usuário e imprime o resultado das três expressões do Desafio 3. Use input() para capturar os valores e mostre cada resultado com descrição clara.

Exemplo:

```
num1 = float(input("Digite o primeiro número: "))
num2 = float(input("Digite o segundo número: "))
num3 = float(input("Digite o terceiro número: "))

exp1 = (num1 > num2) and (num3 >= 10)
print(f"Expressão 1: {exp1}")
```

Compartilhe sua solução no fórum da turma!

Próxima Aula

Estruturas de controle: if, elif, else para criar programas que tomam decisões complexas baseadas em múltiplas condições.

Motivação para Continuar

Dominar estas bases de operadores e lógica coloca você entre os **30% dos desenvolvedores** que consideram Python a linguagem mais produtiva em 2024, segundo o Stack Overflow Developer Survey. Python continua sendo a linguagem mais amada pela comunidade, com taxa de satisfação de 77% entre profissionais.

Seu progresso: Com este conhecimento, você está preparado para construir programas que tomam decisões inteligentes, processam dados com lógica condicional e resolvem problemas do mundo real. Continue praticando!

Made with GAMMA